claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-a0e46aa118f22652c.jsonl`
- SHA-256: `e0deb6b8b39989149993782a4a426f1bfa2ca3baeeab5ea8d719dc65106657d9`
- Source modified: `2026-06-14T06:34:32+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_98c1f280-b9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:32:15.329Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:
- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.
Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.
FORWARD-LOOKING (design confirmation only ? do NOT build it): the owner wants a future "shuttle dropped / shuttle-on-floor for a contiguous frame sequence" signal (service-fault / dead-rally cue). Read fusion_features.py (the compute_*_features + *_FEATURE_NAMES pattern) and backend/eval/experiment.py (Variant.feature_names) and how fusion_golden assembles per-window feature dicts. CONFIRM the architecture cleanly supports adding such a signal as ANOTHER default-OFF feature column (a new compute_shuttle_floor_features + SHUTTLE_FLOOR_FEATURE_NAMES added to a Variant's feature_names) with NO special-casing. present/retained describe whether the seam exists; note exactly where the new feature would plug in.

2. assistant (2026-06-14T06:32:17.744Z)

I'll audit the **shuttle/person/audio fusion signal** component. Let me start by reading the key files mentioned in the task.

3. user (2026-06-14T06:32:21.340Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-14T06:32:21.471Z)

backend\pipeline\detectors\fusion_features.py

5. user (2026-06-14T06:32:21.472Z)

backend\eval\experiment.py

6. user (2026-06-14T06:32:21.478Z)

backend\eval\fusion_golden.py

7. assistant (2026-06-14T06:32:24.821Z)

Note: the `git pull` failed ("Cannot fast-forward to multiple branches") ? a local config quirk, so I'm proceeding read-only on the local checkout. Let me read the three core files.

8. user (2026-06-14T06:32:25.307Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33          if polygon is None or len(polygon) < 3:
34              return False
35          x, y = point
36          inside = False
37          n = len(polygon)
38          j = n - 1
39          for i in range(n):
40              xi, yi = polygon[i]
41              xj, yj = polygon[j]
42              # Does the horizontal ray at y cross edge (i, j)?
43              if (yi > y) != (yj > y):
44                  x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                  if x < x_cross:
46                      inside = not inside
47              j = i
48          return inside
49
50
```

```
    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
        point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
        if court_polygon is None:
            return True
        if frame_width <= 0 or frame_height <= 0:
            return False
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
```

```
                            frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, with x
        normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
        not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
```

```
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174         """
175         if not per_frame or frame_width <= 0 or frame_height <= 0:
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                    for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
```

9. user (2026-06-14T06:32:26.388Z)

```
1       """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3       Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4       candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5       EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6       writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7       CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8       measured by the shipped experiment harness; person features enter only as feature
columns the
```

```
9       learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12          python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31
32      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38      def _derive_video_path(spec: Dict[str, Any]) -> str:
39          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
40          golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming
convention)."""
41          if spec.get("video_path"):
42              return str(spec["video_path"])
43          labels = str(spec["labels"])
44          if labels.endswith(".rallies.csv"):
45              return labels[: -len(".rallies.csv")] + ".MP4"
46          raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
47
48
49      def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
50                        frame_skip: int = 3, iou: float = 0.5,
51                        log_every_sec: float = 15.0) -> Dict[str, Any]:
52          """One golden video -> a feature record (shuttle + person features per candidate
window).
53
54          Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}``.
55          `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
56          when None a real `PersonDetector` is built (loads torchvision on first use).
57          """
58          fps, fw = float(spec["fps"]), float(spec["frame_width"])
59          traj = str(spec["trajectory"])
60          points = TrackNetRunner.parse_trajectory_csv(traj)
61          intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
62          gts = load_golden_gts(str(spec["labels"]))
63          labels = label_candidates(intervals, gts, iou)
64          # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
```

```
estimate
65          # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
66          axis, net_px, _span = rally_gate.estimate_play_axis(points)
67          video_path = _derive_video_path(spec)
68
69          if detector is None:
70              from backend.pipeline.detectors.person_detector import PersonDetector
71              detector = PersonDetector({"indexing": {"use_gpu": True}})
72
73          # Single-pass person detection over ALL candidate windows (open the video ONCE, read
74          # forward ? no per-window seek). Then compute features per window (pure, no GPU).
75          n = len(intervals)
76          win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
77          print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
78          t0 = time.monotonic()
79          state = {"last": t0}
80
81          def _progress(done: int, total: int) -> None:
82              now = time.monotonic()
83              if now - state["last"] >= log_every_sec or done >= total:
84                  el = now - t0
85                  rate = done / el if el > 0 else 0.0
86                  eta = (total - done) / rate if rate > 0 else 0.0
87                  pct = 100 * done // total if total else 0
88                  print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
89                        f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
90                  state["last"] = now
91
92          per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
93                                                        progress_cb=_progress)
94
95          candidates: List[Dict[str, Any]] = []
96          for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
97              det = per_window[i]
98              det_fw = det.get("frame_width") or fw
99              det_fh = det.get("frame_height") or 0.0
100             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101             # normalisation fusion_features uses (x/width, y/height).
102             if axis == "x":
103                 net_norm = net_px / det_fw if det_fw > 0 else 0.5
104             else:
105                 net_norm = net_px / det_fh if det_fh > 0 else 0.5
106             sf, ef = win_frames[i]
107             shuttle_in = [p for p in points if sf <= p.frame <= ef]
108             person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                                 court_polygon=None, net=(axis, net_norm))
110             candidates.append({
111                 "start": float(s), "end": float(e),
112                 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
113                 "person": {k: float(v) for k, v in person.items()},
114                 "label": int(labels[i]),
115             })
116         return {
117             "name": spec["name"], "fps": fps, "frame_width": fw,
118             "candidates": candidates,
119             "gts": [[float(a), float(b)] for a, b in gts],
120         }
121
122
123     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124             fresh: bool = False, smallest_first: bool = False) -> List[str]:
```

```
125         with open(manifest_path, encoding="utf-8") as f:
126             specs = json.load(f)
127         if smallest_first:
128             # Process small videos first for quick feedback (the slow full match lands
last);
129             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
132         os.makedirs(out_dir, exist_ok=True)
133         written: List[str] = []
134         for vi, spec in enumerate(specs, 1):
135             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
136             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
137             # run left off instead of re-detecting). --fresh forces a redo.
138             if not fresh and os.path.isfile(out):
139                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
140                       f"-> {out} (use --fresh to redo)", flush=True)
141                 written.append(out)
142                 continue
143             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
144             t0 = time.monotonic()
145             data = extract_video(spec, frame_skip=frame_skip, iou=iou)
146             n_pos = sum(c["label"] for c in data["candidates"])
147             with open(out, "w", encoding="utf-8") as f:
148                 json.dump(data, f, indent=2)
149             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
150                   f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
151                   f"-> {out}", flush=True)
152             written.append(out)
153         return written
154
155
156     def main() -> None:
157         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172
```

10. user (2026-06-14T06:32:26.705Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
```

```
    identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
    no-regression gate, the pinnable classifier artifact, the persisted-candidate
    substrate) already exists ? this module composes it. **No new scoring math lives
    here**; everything defers to:

      - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
      - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
      - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
      - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
      - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
      - `backend.eval.calibration`    ? `pct_improvement`

    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
    (per-frame signals ? run once on the GPU/WSL box, persisted into
    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
    given persisted candidate features it produces `List[Interval]` deterministically,
    with no GPU and zero production risk. So most experiments are pure offline
    re-scoring of stored data and never touch the served pipeline.

    Three modes (`EXPERIMENT_HARNESS.md` §5):
      - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
        held-out loop, but variant-parameterised.
      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
        human spot-checks (and what two highlight reels would show side by side).
      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
        (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


```

```
    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

```

```
        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
```

```
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
```

```
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
```

```
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
334                     assert model is not None                       # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo:                          # honest=False ? one model over all
videos
337                 model = all_model
338             else:                                      # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                  # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
```

```
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                              "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
```

```
        existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
        shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
        bare LOVO mean at n?6 is not trustworthy on its own.
        """
        if len(videos) < 1:
            raise ExperimentError("compare needs at least one labeled video")
        eval_a = evaluate_variant(videos, variant_a, iou, honest)
        eval_b = evaluate_variant(videos, variant_b, iou, honest)

        delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
        delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured
against
        # different labels" the same way regression.gt_hash guards the no-regression
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------------ SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
```

```
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
    the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
```

```
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                               gts: Optional[List[Interval]] = None,
545                               feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

11. assistant (2026-06-14T06:32:31.407Z)

I have the three core files. Now let me verify the default-OFF claims in the served/production path. I need to check: the default segmenter, whether `cue_flags`/fusion features are wired into production scoring, and whether `fusion_compare.py` exists and how it consumes these features. Let me also confirm the producer (`PersonDetector.detections_over_windows`) and the `distill_local.FEATURE_NAMES`.

12. user (2026-06-14T06:32:31.488Z)

backend\eval\fusion_compare.py

13. user (2026-06-14T06:32:32.942Z)

```
config.json:26:        "default_segmenter": "gemini",
README.md:5:The pipeline is built around three pluggable registries ? **validators**,
**segmenters**, and **AI providers** ? plus a **sport profile** registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
README.md:55:> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster
R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The
shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
README.md:122:You can adjust the parameters of your validators, segmenters, AI provider, and
```

stitcher by editing `config.json` in the root folder. The default config emitted by `setup.py`:
README.md:142:   "default_segmenter": "gemini",
README.md:180:*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:188:[Omitted long matching line]
README.md:242:   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.*
README.md:246:[Omitted long matching line]
README.md:264:   "default_segmenter": "gemini",
README.md:272:[Omitted long matching line]
run_phase3_eval.py:53:   p.add_argument("--segmenter", default="motion",
run_phase3_eval.py:54:   help="Segmenter to run (default: motion ? free/CPU). "
docs\CODE_MAP.md:31: -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:89: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `'motion'`.
docs\CODE_MAP.md:99:[Omitted long matching line]
docs\CODE_MAP.md:105:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
`indexing.default_segmenter='gemini'` diverges from model default `'yolo_hybrid'` ? file
wins at runtime.
docs\CODE_MAP.md:113:[Omitted long matching line]
docs\CODE_MAP.md:114:[Omitted long matching line]
docs\CODE_MAP.md:117:[Omitted long matching line]
docs\CODE_MAP.md:118:[Omitted long matching line]
docs\CODE_MAP.md:127:[Omitted long matching line]
docs\CODE_MAP.md:129:[Omitted long matching line]
docs\CODE_MAP.md:131:[Omitted long matching line]
docs\CODE_MAP.md:142:- `@backend/eval/training.py` ? learned-segmenter data layer.
`::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB;
missing fields default 0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
docs\CODE_MAP.md:144:[Omitted long matching line]
docs\CODE_MAP.md:185:[Omitted long matching line]
docs\CODE_MAP.md:186:[Omitted long matching line]
docs\CODE_MAP.md:226:[Omitted long matching line]
docs\CODE_MAP.md:250:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'`
getattr fallback noted above.
docs\CODE_MAP.md:276:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
run_process_and_stitch.py:74:   seg_name = cfg.indexing.default_segmenter
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No
`min_crossings` knob yet** (see fusion P2).
docs\EXPERIMENT_HARNESS.md:123:JSON-serializable; selected via the existing `segmenter_registry`
+ `IndexingConfig.default_segmenter`.
docs\EXPERIMENT_HARNESS.md:155:pinned ? **rollback = flip `default_segmenter`/variant config,
never a code revert.**
docs\EXPERIMENT_HARNESS.md:185:3. **Per-cue enable flags** in `IndexingConfig` (default
`False`); `default_segmenter` already exists.
backend\reporting\spec.yaml:125: # The segmenter that ran differs from config's default -> you
may be looking at a
backend\reporting\spec.yaml:133:   The segmenter that ran differs from config.json's
default_segmenter, and no
backend\reporting\spec.yaml:134:   explicit --segmenter was given ? so a fallback default
kicked in. The indexer's

[Showing results with pagination = limit: 40]

14. user (2026-06-14T06:32:33.951Z)

```
Found 17 files
docs\QUALITY_ITERATIONS.md
docs\EXPERIMENT_HARNESS.md
tests\test_fusion_compare.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
docs\ANALYZERS_AND_RECONCILERS.md
tests\test_audio_features.py
backend\pipeline\detectors\audio_features.py
tests\test_fusion_hybrid.py
tests\test_fusion_features.py
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
```

15. assistant (2026-06-14T06:32:38.638Z)

There's a `fusion_hybrid.py` segmenter and `fusion_audio.py` too. Critical question: is the fusion segmenter registered and could it become a default? Let me check the production wiring ? `fusion_hybrid.py`, how it's registered, whether `cue_flags` or person features touch the served path, and confirm nothing deletes the code. Let me also check `fusion_compare.py` to see how it consumes the feature dicts.

16. user (2026-06-14T06:32:40.043Z)

```
1       """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3       The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4       and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5       no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6       designed seam): the person features are extra columns the learned LogisticRegression
weights,
7       so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9       Run (no GPU):
10          python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
```

```
        by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
        by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
        vids = [p["video"] for p in res["variant_a"]["per_video"]]
        return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])


    def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
        """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
        feature columns + golden gts). The harness derives labels from gts itself."""
        videos: List[experiment.VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_FEATURE_NAMES:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_FEATURE_NAMES:
                features[fn] = [float(c["person"][fn]) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                     features=features, gts=gts))
        return videos


    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                threshold: float = 0.5) -> Dict[str, Any]:
        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
                                          threshold=threshold)
        fusion = experiment.Variant(name="shuttle_person_fusion",
                                    feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
                                    threshold=threshold)
        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)


    def format_result(res: Dict[str, Any], iou: float) -> str:
        a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
        ds = delta_stats(res)
        st = ds["sign_test"]
        lines = [
            f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
            f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
            f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
            f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
            f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
            f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
            f"| CI clears 0: {ds['ci_excludes_zero']}",
            "  per-video ?F1:",
        ]
        for pv in res["per_video_delta"]:
            lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
        return "\n".join(lines)


    def main() -> None:
        ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
```

```
86          ap.add_argument("--features-dir", default="output")
87          ap.add_argument("--iou", type=float, default=0.5)
88          ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89          args = ap.parse_args()
90
91          videos = load_videos(args.features_dir)
92          if len(videos) < 2:
93              raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94          res = compare(videos, iou=args.iou)
95          print(format_result(res, args.iou))
96          if args.record:
97              row = experiment.record_experiment(res)
98              print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")
99
100
101     if __name__ == "__main__":
102         main()
103
```

17. user (2026-06-14T06:32:40.192Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
```

```
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff


    class FusionSegmenter(TrajectoryHybridSegmenter):
        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
            self._person: Optional[Any] = None
            # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
            # over the whole trajectory once per candidate window (it depends only on `points`).
            self._axis_cache_key: Optional[int] = None
            self._axis_cache: Optional[tuple] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                    "AI provider sets semantic boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
```

```
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          def _axis_estimate(self, points: List[Any]) -> tuple:
93              """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94              computed once per video, not once per candidate window (it depends only on
points)."""
95              key = id(points)
96              if self._axis_cache_key != key or self._axis_cache is None:
97                  self._axis_cache_key = key
98                  self._axis_cache = rally_gate.estimate_play_axis(points)
99              est = self._axis_cache
100             assert est is not None
101             return est
102
103         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                              frame_width: float, video_path: str,
105                              fcfg: Dict[str, Any]) -> Dict[str, float]:
106             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107             frame range and compute the scale/translation-invariant person features. Lazily
builds
108             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110             from backend.pipeline.detectors.person_detector import PersonDetector
111             if self._person is None:
112                 self._person = PersonDetector(self.config)
113
114             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119             fw = det["frame_width"] or frame_width
120             fh = det["frame_height"]
121             # If we can't trust the frame dims, every normalised feature would silently
collapse
122             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123             if fw <= 0 or fh <= 0:
124                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126             poly = fcfg.get("court_polygon")
127             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                               court_polygon=polygon, net=net)
132
133         def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                                 window_stats: List[Dict[str, Any]],
135                                 idx_cfg: Dict[str, Any]) -> List[int]:
136             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137             (0) keep every window ? identical to the substrate. Persisted candidates are
```

```
138             untouched (full superset)."""
139             fcfg = idx_cfg.get("fusion", {})
140             min_crossings = int(fcfg.get("min_crossings", 0))
141             min_players = int(fcfg.get("min_players", 0))
142             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144             keep: List[int] = []
145             for i, st in enumerate(window_stats):
146                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                     continue
148                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                     continue
150                 keep.append(i)
151             return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

18. assistant (2026-06-14T06:32:47.033Z)

Now let me verify the remaining claims: (1) the config schema ? is `fusion.cue_flags.person` default False in the Pydantic config and does `config.json` not select `fusion_hybrid`; (2) the producer `PersonDetector.detections_over_windows` exists; (3) confirm no code deletes/disables fusion because of n=6 ?F1. Let me check the config models, the audio cue variant (the third fusion signal), and git log for the "non-significant ?F1" commit.

19. user (2026-06-14T06:32:49.005Z)

```
78:class FusionConfig(BaseModel):
79:    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
81:    Every default reproduces control behaviour: the fusion segmenter persists the
82:    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
83:    true ? the person-cue features, but it does NOT gate the served handoff unless a
84:    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
86:    supplied ? off-court persons (spectators / adjacent courts) are excluded.
90:    # Windowing velocity threshold for the fusion family (the engine reads
92:    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
94:    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
96:    cue_flags: dict[str, bool] = Field(default_factory=dict)
99:    min_crossings: int = Field(default=0, ge=0)
100:    # Served Gate B: when the person cue is on, drop windows with median in-court players
102:    min_players: int = Field(default=0, ge=0)
105:    # Net line for the person two-sided-motion split (person features only ? the shuttle
142:    fusion: FusionConfig = Field(default_factory=FusionConfig)
```

20. user (2026-06-14T06:32:50.411Z)

```
258:    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
305:    def detections_over_windows(self, video_path: str, windows: List[Tuple[int, int]],
```

21. user (2026-06-14T06:32:51.855Z)

```
44:            c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
68:        for fn in af.AUDIO_FEATURE_NAMES:
84:        feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
list(af.AUDIO_FEATURE_NAMES),
```

22. user (2026-06-14T06:32:52.696Z)

```
40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
116:            model = LogisticRegression().fit(X, y, FEATURE_NAMES)
```

```
135:    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
137:    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
```

23. assistant (2026-06-14T06:32:58.307Z)

The audio cue (`fusion_audio.py`) already demonstrates the exact extension pattern: `af.AUDIO_FEATURE_NAMES` appended to `feature_names` as a third column block ? the precise seam the forward-looking shuttle-floor signal would use. Let me confirm: (1) the FusionConfig `cue_flags` default and the audio module's `*_FEATURE_NAMES` + `compute_*` pattern; (2) `config.json` doesn't select `fusion_hybrid`; (3) git log evidence that the n=6 ?F1 was non-significant and nothing was deleted.

24. user (2026-06-14T06:32:59.961Z)

```
78      class FusionConfig(BaseModel):
79          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81          Every default reproduces control behaviour: the fusion segmenter persists the
82          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83          true ? the person-cue features, but it does NOT gate the served handoff unless a
84          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86          supplied ? off-court persons (spectators / adjacent courts) are excluded.
87          """
88          # Which trajectory substrate seeds the windows (shuttle stays primary).
89          substrate: Literal["wasb", "tracknet"] = "wasb"
90          # Windowing velocity threshold for the fusion family (the engine reads
91          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93          velocity_threshold: float = 0.02
94          # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95          # enabled ? so the default path never imports torch / touches the GPU.
96          cue_flags: dict[str, bool] = Field(default_factory=dict)
97          # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98          # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99          min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08
115         min_segment_duration: float = 3.0
116         dead_time_merge_gap: float = 2.0
117         chunk_duration: float = 90.0
118         chunk_overlap: float = 10.0
119         detector_model: str = "fasterrcnn_resnet50_fpn"
120         detector_score_threshold: float = 0.5
121         yolo_velocity_threshold: float = 0.15
```

```
122         yolo_frame_skip: int = 3
```

25. user (2026-06-14T06:33:01.040Z)

```
1       """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3       Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4       `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5       window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6       and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7       of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8       (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10      Run (no GPU):
11          python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12      """
13      from __future__ import annotations
14
15      import argparse
16      import glob
17      import json
18      import os
19      from typing import Any, Dict, List
20
21      from backend.eval import experiment
22      from backend.eval.fusion_compare import delta_stats
23      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24      from backend.pipeline.detectors import audio_features as af
25      from backend.pipeline.audio.hit_detector import detect_hits_in_video
26
27
28      def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
29          """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
30          with open(manifest_path, encoding="utf-8") as f:
31              specs = json.load(f)
32          written: List[str] = []
33          for spec in specs:
34              jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
35              if not os.path.isfile(jp):
36                  print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
37                  continue
38              video = _derive_video_path(spec)
39              print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
40              hits = detect_hits_in_video(video, k=k)
41              with open(jp, encoding="utf-8") as f:
42                  data = json.load(f)
43              for c in data["candidates"]:
44                  c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
45              with open(jp, "w", encoding="utf-8") as f:
46                  json.dump(data, f, indent=2)
47              print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
48              written.append(jp)
49          return written
```

```
50
51
52      def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
53          """Load the feature JSONs into VideoCandidates including the audio columns (requires
54          `augment` to have run ? raises if a JSON lacks audio)."""
55          videos: List[experiment.VideoCandidates] = []
56          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
57              with open(path, encoding="utf-8") as f:
58                  d = json.load(f)
59              cands = d["candidates"]
60              if cands and "audio" not in cands[0]:
61                  raise SystemExit(f"{path} has no audio features ? run `--augment` first")
62              intervals = [(float(c["start"]), float(c["end"])) for c in cands]
63              features: Dict[str, List[float]] = {}
64              for fn in SHUTTLE_FEATURE_NAMES:
65                  features[fn] = [float(c["shuttle"][fn]) for c in cands]
66              for fn in PERSON_FEATURE_NAMES:
67                  features[fn] = [float(c["person"][fn]) for c in cands]
68              for fn in af.AUDIO_FEATURE_NAMES:
69                  features[fn] = [float(c["audio"][fn]) for c in cands]
70              gts = [(float(a), float(b)) for a, b in d["gts"]]
71              videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
72                                                       features=features, gts=gts))
73          return videos
74
75
76      def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
77                            threshold: float = 0.5) -> Dict[str, Any]:
78          """Incremental audio lift: shuttle+person  vs  shuttle+person+audio (honest
LOVO)."""
79          person = experiment.Variant(
80              name="shuttle_person",
81              feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
threshold=threshold)
82          person_audio = experiment.Variant(
83              name="shuttle_person_audio",
84              feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
list(af.AUDIO_FEATURE_NAMES),
85              threshold=threshold)
86          return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
87
88
89      def format_result(res: Dict[str, Any], iou: float) -> str:
90          a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
91          ds = delta_stats(res)
92          st = ds["sign_test"]
93          lines = [
94              f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
honest) ===",
95              f"  shuttle+person       : F1={a['f1']:.3f}  P={a['precision']:.3f}
R={a['recall']:.3f}",
96              f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
R={b['recall']:.3f}",
97              f"  incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
98              f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
99              f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
100             f"| CI clears 0: {ds['ci_excludes_zero']}",
101         ]
102         for pv in res["per_video_delta"]:
103             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
104         return "\n".join(lines)
105
106
107     def main() -> None:
```

```
108         ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the
golden set.")
109         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
110         ap.add_argument("--features-dir", default="output")
111         ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
112         ap.add_argument("--augment", action="store_true", help="add audio features to the
existing JSONs")
113         ap.add_argument("--compare", action="store_true", help="report the incremental audio
LOVO ?F1")
114         ap.add_argument("--iou", type=float, default=0.5)
115         ap.add_argument("--record", action="store_true")
116         args = ap.parse_args()
117
118         if args.augment:
119             augment(args.manifest, args.features_dir, args.k)
120         if args.compare:
121             videos = load_videos_with_audio(args.features_dir)
122             if len(videos) < 2:
123                 raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
124             res = compare_increment(videos, iou=args.iou)
125             print(format_result(res, args.iou))
126             if args.record:
127                 row = experiment.record_experiment(res)
128                 print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")
129
130
131     if __name__ == "__main__":
132         main()
133
```

26. user (2026-06-14T06:33:01.673Z)

```
15:AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]
18:def compute_audio_features(hits: List, window_start: float, window_end: float) -> Dict[str,
float]:
28:        return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
```

27. assistant (2026-06-14T06:33:05.485Z)

This confirms the architecture seam precisely. Now let me verify the final claims: (1) `config.json` does not select `fusion_hybrid` and the on-disk fusion config; (2) git-log evidence that the n=6 ?F1 was non-significant and that nothing was deleted/disabled in response. Let me check the leaderboard and the QUALITY_ITERATIONS doc for the actual measured result, plus the config.json fusion block.

28. user (2026-06-14T06:33:09.714Z)

```
=== config.json default_segmenter + fusion block ===
default_segmenter: gemini
fusion in indexing: False
fusion block: "<ABSENT>"
=== leaderboard ===
-rw-r--r-- 1 avidu 197609 2506 Jun 14 11:54 eval_baselines/experiment_leaderboard.json
=== git log fusion/delta-f1 ===
2e5892e feat(eval): fix the learned variant ? train-fold threshold tuning + calibration (C2)
(#130)
f680b8d feat(eval): wire C1+C4 honest reporting into experiment.compare() (additive) (#129)
d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
dc19bf2 feat(eval): fusion P4 ? audio cue + incremental-lift measurement (default-OFF) (#122)
39e961b feat(eval): fusion P3 measurement ? golden person-feature extractor + LOVO compare
(#121)
cbf2236 feat(segmenters): FusionSegmenter ? shuttle + net-crossing + person-cue fusion (P2,
default-OFF) (#117)
```

ecee786 feat(eval): experiment harness ? A/B + SxS variant layer (no-regression by design)
(#114)

29. assistant (2026-06-14T06:33:13.846Z)

All claims confirmed. `default_segmenter` is `gemini` (and the model default is `yolo_hybrid`);
`fusion_hybrid` is never selected, and there's no `fusion` block in `config.json` (so
`cue_flags`/`min_*` all take their default-OFF values). Let me check the leaderboard content for
the measured non-significant ?F1 and confirm no commit deletes the fusion code.

30. user (2026-06-14T06:33:15.111Z)


```
[
  {
    "timestamp": "2026-06-14T06:23:51.507393+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
      "name": "shuttle_only",
      "spec_hash": "47c867c36f2f",
      "aggregate": {
        "precision": 0.31807603780927995,
        "recall": 0.3101947096042718,
        "f1": 0.3068611057861412,
        "mean_iou": 0.5791831889148886
      }
    },
    "variant_b": {
      "name": "shuttle_person_fusion",
      "spec_hash": "253f4709b4c4",
      "aggregate": {
        "precision": 0.3617228001510626,
        "recall": 0.2708939086387243,
        "f1": 0.2863398846047612,
        "mean_iou": 0.5690111885714347
      }
    },
    "delta": {
      "precision": 0.04364676234178266,
      "recall": -0.0393008009655475,
      "f1": -0.02052122118138,
      "mean_iou": -0.01017200034345389
    },
    "honest_delta_f1": {
      "mean_delta": -0.02052122118137999,
      "ci_low": -0.16462806145542527,
      "ci_high": 0.16224446866060574,
      "n": 6.0,
      "ci_excludes_zero": false,
      "sign_test": {
        "wins": 2.0,
        "losses": 4.0,
        "ties": 0.0,
        "n_effective": 6.0,
        "p_value": 0.6875
      }
    }
  },
  {
    "timestamp": "2026-06-14T06:24:56.487711+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
```

```
            "name": "shuttle_person",
            "spec_hash": "ee89f1e41604",
            "aggregate": {
              "precision": 0.3617228001510626,
              "recall": 0.2708939086387243,
              "f1": 0.2863398846047612,
              "mean_iou": 0.5690111885714347
            }
          },
          "variant_b": {
            "name": "shuttle_person_audio",
            "spec_hash": "d68c39eba87c",
            "aggregate": {
              "precision": 0.4530798177857001,
              "recall": 0.3408138120839734,
              "f1": 0.3655826552797436,
              "mean_iou": 0.7074802852140344
            }
          },
          "delta": {
            "precision": 0.0913570176346375,
            "recall": 0.06991990344524907,
            "f1": 0.07924277067498242,
            "mean_iou": 0.13846909664259965
          },
          "honest_delta_f1": {
            "mean_delta": 0.07924277067498244,
            "ci_low": -0.053508315003560535,
            "ci_high": 0.2029045239922802,
            "n": 6.0,
            "ci_excludes_zero": false,
            "sign_test": {
              "wins": 4.0,
              "losses": 2.0,
              "ties": 0.0,
              "n_effective": 6.0,
              "p_value": 0.6875
            }
          }
        }
      ]
```


31. user (2026-06-14T06:33:16.395Z)

```
30:| Gen-0 owned head (LOVO, n=6) | 0.744 | 0.561 |
43:### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising
(not promotable) *(2026-06-14)*
46:test; never a bare mean at n=6)**:
48:| Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears 0 |
54:**Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign tests
n.s.) at n=6 ? **both stay
57:(largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**
shuttle-only (0.307) ? but n=6 can't
64:(?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the
78:(Platt/isotonic, C2). Validated on the n=6 golden set: **mean F1 0.307 ? 0.423 (+0.116)**,
testlarge
85:### First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*
87:**Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p =
0.031**, CI
105:  needs), `paired_delta_ci` (B?A with a CI that must clear 0). At n?6 a lone LOVO mean is
not trustworthy.
184:- **`2026-06-multivideo-lovo/`** ? the **n=6 corpus + honest LOVO**; established the
heuristic is
187:  ~+0.10** (n=6: learned 0.583 vs heuristic 0.480), with **AUC 0.83?0.92 decoupled from
```

contrast**. This

32. assistant (2026-06-14T06:33:23.561Z)

I have everything I need. The leaderboard and `QUALITY_ITERATIONS.md` line 54 confirm: "neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay**" ? i.e., the non-significant ?F1 led to *retention*, not deletion. The most recent git commit (`#130`) still adds capability to the fusion path; nothing removes it.

Let me do one final check to confirm no commit message or recent change disables/deletes the fusion code, and read the full QUALITY_ITERATIONS line 54-65 context for the "both stay" decision.

33. user (2026-06-14T06:33:23.731Z)

```
43      ### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio =
promising (not promotable) *(2026-06-14)*
44      The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ?
`output/*_golden_features.json`, the
45      replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats
(C1 ? bootstrap 95% CI + paired sign
46      test; never a bare mean at n=6)**:
47
48      | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears
0 |
49      |---|---|---|---|---|---|
50      | shuttle-only | 0.307 | ? | ? | ? | ? |
51      | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) |
**No** |
52      | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L
(p=0.688) | **No** |
53
54      **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign
tests n.s.) at n=6 ? **both stay
55      default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?
`players_in_court` counts spectators);
56      per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising
lead ? +0.079 / +27.7%, wins 4/6
57      (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above**
shuttle-only (0.307) ? but n=6 can't
58      confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and
measure audio on shuttle-only
59      *directly* (without the person handicap). Recorded in
`eval_baselines/experiment_leaderboard.json`. **Negative/uncertain
60      results are kept on purpose ? this is the honest ablation backbone for the paper aim.**
61
62      ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*
63      **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The
measured win
64      (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the
65      [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
clears the honest
66      bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the
eval-only Gate A
67      folded into production, MULTI_SIGNAL_FUSION_PLAN §3.1). ? The offline win is on the
recall-oriented *proposal*
```

34. user (2026-06-14T06:33:26.477Z)

```
=== any deletions/disables of fusion since the n=6 measurement (#127)? ===
9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus
fa12829 chore: gitignore scratch/ to keep git status clean (#136)
```

```
55df82f feat(obs): run-telemetry black box ? fps + GPU temp/clocks/power + CPU/RAM (rotating
~10KB JSONL) (#134)
a9432a9 feat(db): self-scheduling job substrate ? migration v3 + claim_due_job (P2) (#135)
ec2002d feat(infra): ExecutionPolicy P1 ? bound the Gemini hang (op_timeout + bounded poll)
(#133)
2673eee docs: execution-policy framework ? per-job, self-scheduling resilience (design) (#132)
c8bdffd docs: promote Gate-A (measured win) + per-video output layout plan (#131)
2e5892e feat(eval): fix the learned variant ? train-fold threshold tuning + calibration (C2)
(#130)
f680b8d feat(eval): wire C1+C4 honest reporting into experiment.compare() (additive) (#129)
8398e52 docs(archive): first end-to-end A/B experiment on the golden corpus (n=6) (#128)
--- diffstat of files touched since #127 ---
 backend/eval/experiment.py | 130 ++++++++++++++++++++++++++++++++++++++++++++---
 1 file changed, 122 insertions(+), 8 deletions(-)
=== confirm producer + feature math still present (grep registry register) ===
backend/pipeline/segmenters/fusion_hybrid.py:154:segmenter_registry.register(FusionSegmenter)
```

35. assistant (2026-06-14T06:33:37.227Z)

All claims are now verified with concrete evidence. The only post-measurement change to the fusion eval code (`#130`) was purely **additive** (+122/?8 lines, the C2 threshold-tuning/calibration fix) ? nothing deleted the producer or feature math. The QUALITY_ITERATIONS doc explicitly states "**Negative/uncertain results are kept on purpose**" and "**both stay default-OFF**".

I have a complete, skeptical picture. Let me record findings.

36. user (2026-06-14T06:34:18.295Z)

Structured output provided successfully

37. assistant (2026-06-14T06:34:32.291Z)

Audit complete. I verified the **multi-signal fusion cue** (shuttle + person + audio) across producers, the served `fusion_hybrid` segmenter, and the offline experiment-harness scoring path. All four properties hold:

- **present** ? `fusion_features.py` (PERSON, 5 cols), `audio_features.py` (AUDIO, 3 cols), the registered `FusionSegmenter`, and the `fusion_golden`/`fusion_compare`/`fusion_audio` measurement drivers all exist.
- **default_off** ? `cue_flags` defaults to `{}` (person OFF), `min_crossings`/`min_players` default 0; `config.json` selects `gemini` and has **no `fusion` block at all**, so `fusion_hybrid` is never the served segmenter and every knob is at its OFF default.
- **no_special_casing** ? person/audio enter the decision purely as feature columns the `LogisticRegression` weights (`experiment.py:227-238` `predict` ? `_feature_matrix(variant.feature_names)`); a cue is added by extending `Variant.feature_names`, not by a per-cue if/else. (The only hard if/else gates ? Gate A net_crossings, Gate B players_in_court ? are the separate, default-0 rally_gate heuristic, not the fusion cue.)
- **retained** ? the n=6 ?F1 was non-significant (person ?0.021, audio +0.079, both CIs span 0) and the explicit decision was "both stay default-OFF"; `QUALITY_ITERATIONS.md` states "Negative/uncertain results are kept on purpose." The only post-measurement fusion change was additive (+122/?8 in `experiment.py`, the C2 fix). Nothing deletes or disables the code.

Forward-looking seam (design-confirmed, not built): a future "shuttle-on-floor for a contiguous frame sequence" signal plugs in cleanly as another default-OFF column via the exact audio-cue pattern ? a new `compute_shuttle_floor_features` + `SHUTTLE_FLOOR_FEATURE_NAMES`, added to a `Variant`'s `feature_names` (extend `fusion_compare.py:60` / `fusion_audio.py:84`). The scorer weights it automatically, and `experiment.py:_feature_column` already defaults a missing column to 0.0 for back-compat. The main risk to flag: implement it as a feature column, **not** as a third hard `_gate_mask` branch like Gate A/B.

Full file:line evidence and concerns are in the StructuredOutput.